How to Use MicroCyte for Cell Cycle Analysis: A Beginner's Guide

Updated 02.14.2025 – S. Perritt

MicroCyte is not a singular program or script – it is a workflow designed to generate datasets from images and help you to analyze the datasets. We will go over how to use these scripts, but first you need to know how to access MicroCyte! You will need RStudio (or another R GUI, but that's your own choice), and the MicroCyte download. You'll also need to download ImageJ if you don't already have it.

To download RStudio:

- Go to https://rstudio-education.github.io/hopr/ and refer to the Appendix section A for instructions on downloading R and RStudio for your device.

To download MicroCyte:

- Go to https://github.com/SThompsonLab.
- Click on the MicroCyte repository.
- Click the "Code" drop down menu and select "Download ZIP".
- Open the ZIP folder and you will find the MicroCyte-main folder. Copy this folder into the lab notebook page for the experiment to be analyzed.

When you first open the MicroCyte folder, you should see four files:

- bin: folder containing all the scripts that will be sourced during the workflow (and more!)
- microCyte: R file
- README: MD file with some basics on how to use the package
- schema: .csv file which will be filled in with information about the experiment

The basic MicroCyte workflow is as follows:

1. Build schema file.
2. Generate image directories (dirGen).
3. Take images and save to appropriate directories.
4. Convert .tif images to .png and save originals to a safe folder; purge spectral overlap, if needed (purgo).
5. Rename images to target names as listed in schema (reName).
6. Use ImageJ macro to pull anchor extraction data from .png files (YggData_multi).
7. Combine data from all channels for each image (imaGen).
8. Combine data from all images for each condition (concato).
9. Combine data into a single dataset or multiple datasets based on a variable (unite).
10. Analyze.

So, let's get into it. I will be using the images taken for the experiment Correlating early S phase with TAg expression during a BKPyV-Dunlop infection.

Step 1: Build schema file.

- Open the schema file. It should be mostly empty, with five headers:

notebook_id	CH_1	CH_2	CH_3	CH_4	CH_5

- Populate the schema file with information about your experiment.
    - Each channel should be named for its respective target. Put NA in the column for any unused channels.
    - Each condition (unique combination of replicate, infection, drug treatment, etc) MUST have its own row in the file. For example, below we have Cold v Warm, Mock v BKPyV, Fresh v Conditioned, and three RPTE lines (RL). There is one row in the file for each of these unique conditions.

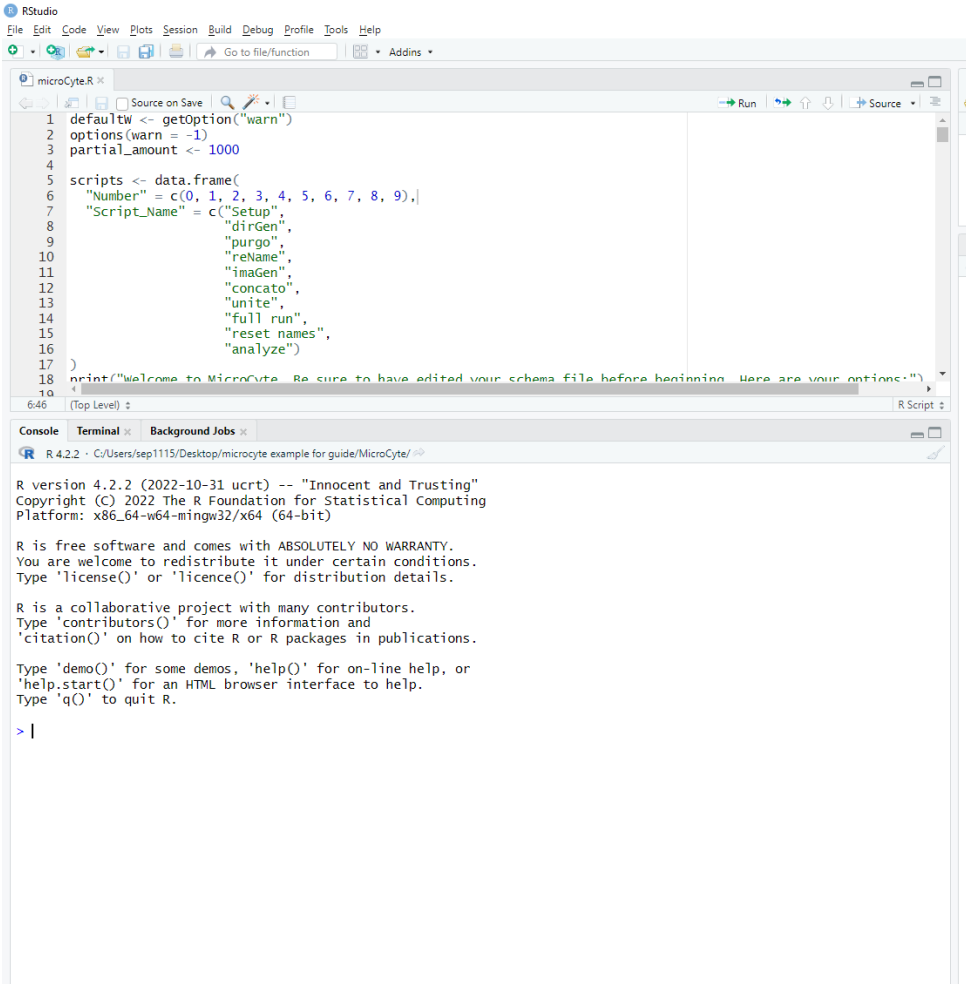
notebook_id	CH_1	CH_2	CH_3	CH_4	CH_5	condition	infection	media	rLine
JNp96.1_1	brdu	tag	edu	dna	NA	Cold	Mock	Fresh	RL6-1
JNp96.1_2	brdu	tag	edu	dna	NA	Cold	Mock	Fresh	RL7-1
JNp96.1_3	brdu	tag	edu	dna	NA	Cold	Mock	Fresh	RL6-2
JNp96.1_4	brdu	tag	edu	dna	NA	Cold	BKPyV	Fresh	RL6-1
JNp96.1_5	brdu	tag	edu	dna	NA	Cold	BKPyV	Fresh	RL7-1
JNp96.1_6	brdu	tag	edu	dna	NA	Cold	BKPyV	Fresh	RL6-2
JNp96.1_7	brdu	tag	edu	dna	NA	Cold	Mock	Conditioned	RL6-1
JNp96.1_8	brdu	tag	edu	dna	NA	Cold	Mock	Conditioned	RL7-1
JNp96.1_9	brdu	tag	edu	dna	NA	Cold	Mock	Conditioned	RL6-2
JNp96.1_10	brdu	tag	edu	dna	NA	Cold	BKPyV	Conditioned	RL6-1
JNp96.1_11	brdu	tag	edu	dna	NA	Cold	BKPyV	Conditioned	RL7-1
JNp96.1_12	brdu	tag	edu	dna	NA	Cold	BKPyV	Conditioned	RL6-2
JNp96.1_13	brdu	tag	edu	dna	NA	Warm	Mock	Fresh	RL6-1
JNp96.1_14	brdu	tag	edu	dna	NA	Warm	Mock	Fresh	RL7-1
JNp96.1_15	brdu	tag	edu	dna	NA	Warm	Mock	Fresh	RL6-2
JNp96.1_1	brdu	tag	edu	dna	NA	Warm	BKPyV	Fresh	RL6-1

6									
JNp96.1_17	brdu	tag	edu	dna	NA	Warm	BKPyV	Fresh	RL7-1
JNp96.1_18	brdu	tag	edu	dna	NA	Warm	BKPyV	Fresh	RL6-2
JNp96.1_19	brdu	tag	edu	dna	NA	Warm	Mock	Conditioned	RL6-1
JNp96.1_20	brdu	tag	edu	dna	NA	Warm	Mock	Conditioned	RL7-1
JNp96.1_21	brdu	tag	edu	dna	NA	Warm	Mock	Conditioned	RL6-2
JNp96.1_22	brdu	tag	edu	dna	NA	Warm	BKPyV	Conditioned	RL6-1
JNp96.1_23	brdu	tag	edu	dna	NA	Warm	BKPyV	Conditioned	RL7-1
JNp96.1_24	brdu	tag	edu	dna	NA	Warm	BKPyV	Conditioned	RL6-2

- Save and close the schema file.

Step 2: Generate image directories (dirGen).

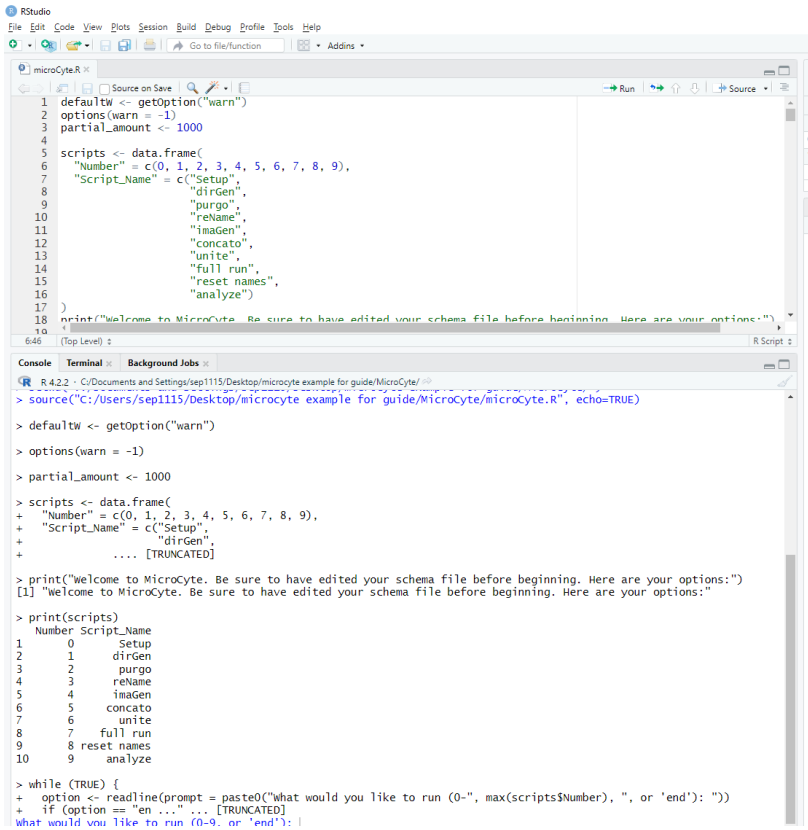
- Click the MicroCyte R file. RStudio should launch with the MicroCyte script in the Source Editor pane.



- Set your working directory to your current MicroCyte folder, if it isn't already:

```
setwd("/Path/to/microcyte/example/MicroCyte/")
```

- Click the "Source" button at the top right of the Source Editor pane. MicroCyte will now be sourced and will load a menu in the Console pane:



- You will be prompted to select an option:

  What would you like to run (0-9, or 'end'):

- If you have never run MicroCyte on your device, type 0 and press enter. The Setup script will run, loading packages necessary for the workflow.
- When prompted again, type 1 and press enter. The dirGen script will now run. You will be asked a few questions, some of which have default options, which are in all caps. You can hit enter to select the default option or type another option and hit enter.

  Generating directories. Should this be done by name or number (NAME/number):

  The answer to this is nearly always NAME. The script will generate a name_id for each row of the schema file.

  Are you taking single images, or automated capture (AUTO/single):

  This answer is nearly always “single”, unless you don’t care about refocusing your images between channels.

  How many images per condition (Default = 5):

  This is up to you – more images generally makes for cleaner/more accurate data. Do at least 5.

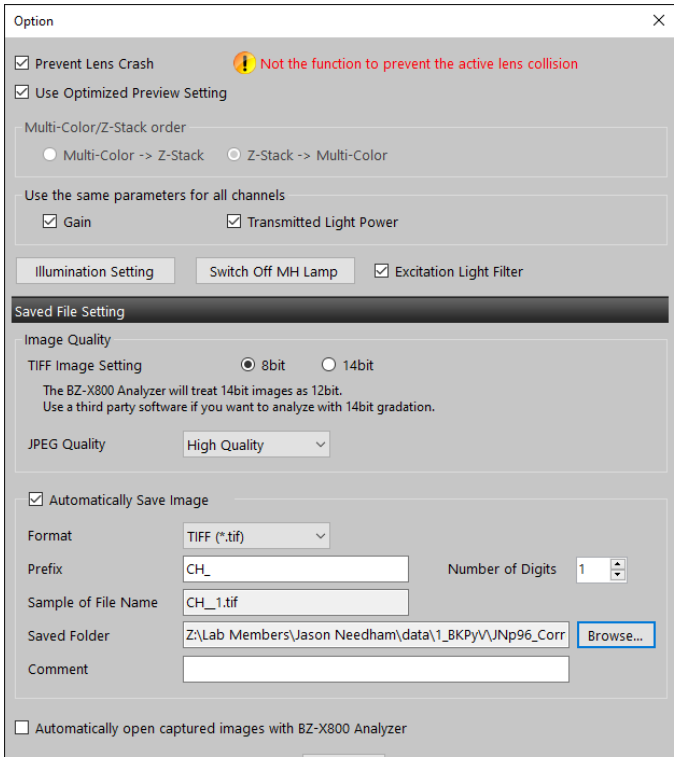
  How many targets are listed in the schema (Default 5):

  How many channels are you using? Jason used 4 in the example we are using: brdu, edu, tag, and dna.
- Your MicroCyte folder will now have three new folders: data, figures, and files.

- The files folder will contain a subfolder for each row of your schema file, and each subfolder will contain image folders (however many images per condition you entered).

Step 3: Take images and save to appropriate directories. **(IF NOT USING A KEYENCE SCOPE, SIMPLY SAVE YOUR IMAGES USING THE NAMING SYSTEM BELOW IN THE DIRECTORIES GENERATED ABOVE)**

- Start up the Keyence scope and load your plate. Use your nuclear stain to get into focus.
    - Make sure the exposure is high enough so that the nuclei are clear and round – a little bit overexposed is better than under. Use the black balance to create sharp contrast between the background and the foci.
    - Select the Menu tab in the top left of the scope software, then select Option. This window will open:



- Make sure that the Prefix box says “CH_”
    - This will ensure that all images are named in a way that matches the schema columns.
- Use the Browse button to navigate to the folder for whichever condition you choose to image first and select the first image folder (“image_1”). Click OK, then close the Options menu.
- Now, take your images. Make sure to follow these guidelines:
    - **Always** take your images in the order listed in the schema. For example, if you have

CH_1	CH_2	CH_3	CH_4
brdu	tag	edu	dna

then you must image the brdu, tag, edu, and dna channels, in that order, every time. Your set prefix will ensure that all images are automatically numbered with the appropriate channel number.

- **Always** make sure your nuclei look sharp: underexposed/clumpy/ugly nuclei will yield ugly data.
- **Never** black balance any channel besides the nuclear anchor.
    - A good rule of thumb: if the question is *yes/no* (is there a nucleus here?), black balance is allowed. If the question is *how much* (how much TAg intensity does each nucleus have?) black balance should be avoided. If you would like both (Where is the nucleus **and** how much DNA is present), take two images – one black balanced for ROI designation and the other without for raw DAPI intensity quantification.

Step 4: Run purgo (Convert .tif images to .png and save originals to a safe folder; purge spectral overlap, if needed).

- Now that your images are taken, you can shut down the scope and return to RStudio.
- If you still have MicroCyte open and sourced, you should still see the prompt in your Console pane asking which script you'd like to run.
    - If not, open and source MicroCyte again.
- Type 2 and hit enter to run the purgo script. Two questions will be asked:

  What should the runtype be (auto/manual/full/none):

  The answer will be "none" unless you have spectral overlap between channels.

  should QC graphs be generated (Y/n):

  If you don't have overlap, select n. If you do, select Y to generate QC graphs which will compare channels.

- Purgo will now run, after which each image folder will have two new folders: originals and PNGS.
    - If you chose to have the overlap purged and/or the QC graphs you will have additional new folders.

Step 5: Run reName to change image names (CH_1, etc) to target names as listed in schema.

- Once purgo is finished, you will automatically be prompted:

  Would you like to now run reName (Y/n)?
- Hit enter to run. Your images will all be renamed according to the schema file.
    - For example, if 'brdu' was the target under the CH_1 column, all images named CH_1 will now be named brdu.
- Once completed, you will be prompted to run YggData before continuing.

Step 6: Use ImageJ macro to pull anchor extraction data from .png files (YggData_multi).

- Open ImageJ (wait 5-7 business days for it to load).
- Click Plugins > Macros > Edit.
- Navigate to your MicroCyte folder, then bin > scripts > scripts. Select and open "4_YggData_multi".
    - A macro editor window will open.
- Drag and drop one of your DNA .png images to open it in ImageJ.
    - *If you try to do this with a .tif file it will not work.*
- You will now perform a series of steps that will tell the macro what to count as a cell:
    - Select Image > Type > 8-bit. The image will turn greyscale.
    - Select Image > Adjust > Threshold. The cells will turn red and a small Threshold window will open.
        - Turn the value up or down until your cells are sharply defined and well separated from the background. Make sure that you don't turn the threshold too high, or you'll begin to lose the edges of some cells. The default value is 45. Hit Apply once you select your value.
        - Type your selected value into the macro editor window beside the anchorMinThreshold variable.
        - You can now close the Threshold window.
    - Select Process > Binary > Watershed.
        - This will draw lines between cells that are close to each other.
    - Select Analyze > Analyze Particles.
        - The Analyze Particles window will open. You must select size and circularity parameters; nuclei which do not fit these parameters will be excluded.
        - The YggData macro's default size is 30-400 pixels^2 and the default minimum circularity is 0.75.
        - You can play around with different values to see what gets the most accurate counts for your image. Check the Display Results box and hit OK. Your results will be displayed by putting a number on each nucleus that was counted – the nuclei which do not fit the parameters will remain uncounted.
        - Once you find the right parameters, input these on the macro editor for the anchorSize and circularity variables.
    - Click Run, then select any .png image in your dataset.
    - The macro will now violently iterate through all of your images.
        - If you are prone to seizures, step away for 2-5 minutes.
    - After running, each image folder will have a new folder named. Anchor_Extraction, which will contain one .csv file for each channel.
- You may close out of all ImageJ windows now and return to RStudio.

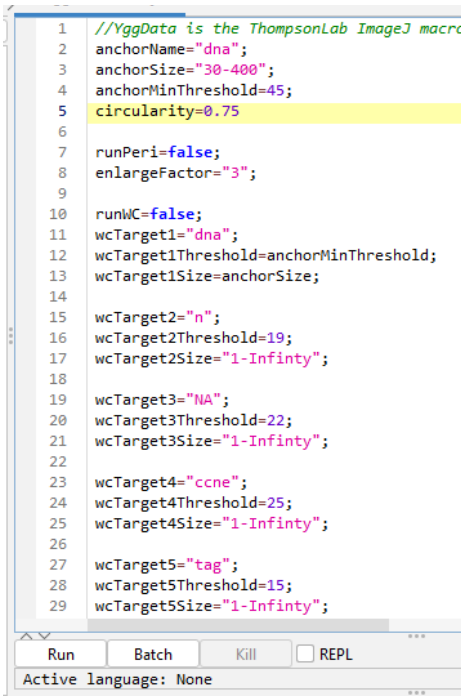


Figure 1: Macro Editor with default parameters.

Step 7: Run ImaGen to combine data from all channels for each image.

- On RStudio, type 4 and hit enter to begin running imaGen.
- The script will begin iterating through each image folder and combining the .csv files for each channel into one.
- Once completed, each image folder will contain a new file named “image_x_WN_all.csv”

Step 8: Run concato to combine data from all images for each condition.

- After imaGen runs, you will be prompted to run concato() to bring all ROI data from individual images into a single dataset for the sample.
- You will be asked:

  Should the numeric values be normalized among image sets (Y/n)?

  This is up to you. If you select yes, the script will normalize (lower quartile) the values between images in each condition folder to account for variability between images (see README file for a more technical explanation).

- Each condition folder will now contain a combined .csv file.

Step 9: Run unite to combine datasets.

- After concato is done, you will be prompted to run unite. Do that.
- You will be asked:

  Should the entire dataset be used (FULL), or equivalent cells per sample (partial):

  Again, up to you. If you pick the default, all data will be included. If you pick “partial”, you will only include data for an equal number of cells per condition.

  Should samples be united on a variable (VAR), or altogether (all):

  Fairly self-explanatory – you can choose to combine all the data into a single file (all) or multiple files based on a variable, like infection or drug treatment. If you choose VAR, you will then be asked:

  Is the variable a part of the (name), or a (SCHEMA) variable:

  I always pick SCHEMA here since everything in the file name is contained in the schema file too.

  Which shema variable would you like to unite by:

  You will be presented with a list of the column names in your schema file. Type one and hit enter, then unite will run. Your “data” folder is no longer empty! Now you have data, and you must analyze it.

If you haven't run a MicroCyte analysis on your device before, run menu option 9 (analyze) to load a whole bunch of useful packages and scripts.

Step 10: Analyze

So, this is where things become more of a choose-your-own-adventure. Regardless of what you choose to do, you should save your work:

Click File > New File > R script. A new tab should open in the Source Editor pane. This is where you will type your analysis script so it can be saved for later.

Step 10a: Run sirmixaplot/set thresholds for EdU, TAg, etc

- If you are doing a typical cell cycle analysis, then you'll want to start out by running SirMixAPlot:
    - Type sirmixaplot("data/Experiment_all.csv") or whatever your file name is.
    - Run this line to start the script. The console will print a list of column names in your file and ask:

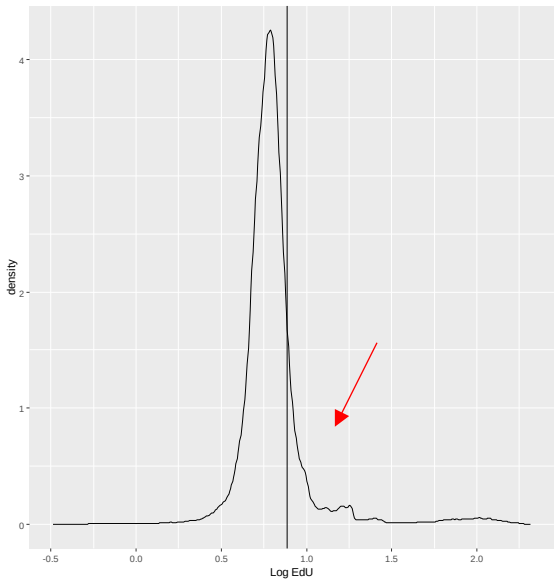
      px - What parameter is the x-axis:

      The script is attempting to build a dot plot to determine cell cycle states. You must tell it what parameters to build the plot with. I usually type log2_dna (log2 transformed DNA intensity).

      What is the name of the x axis:

      Type whatever axis label you'd like to appear on the dot plot.

      You will also be prompted to select a y axis variable and label. I typically use Mean_ANC_edu (mean EdU intensity across the anchor ROI).
- The script will now generate a dot plot of DNA vs EdU intensity.
- Next, it should generate a line graph that looks something like this:
    - Your plot will differ some, but you should see a large peak (G1 cells) with a small hump (S phase cells) beside it.
    - There is a vertical line automatically drawn where the script believes the cutoff between EdU positive/negative cells should be. You will be asked to confirm whether this cutoff is correct: Is 0.89 representative of the EdU cutoff? (Y/n)
    - It is not. Your cutoff should be at the base of the large peak and before the hump (cutoff at red arrow). Enter "n" and then input the appropriate value (here, ~1-1.1 would be good).

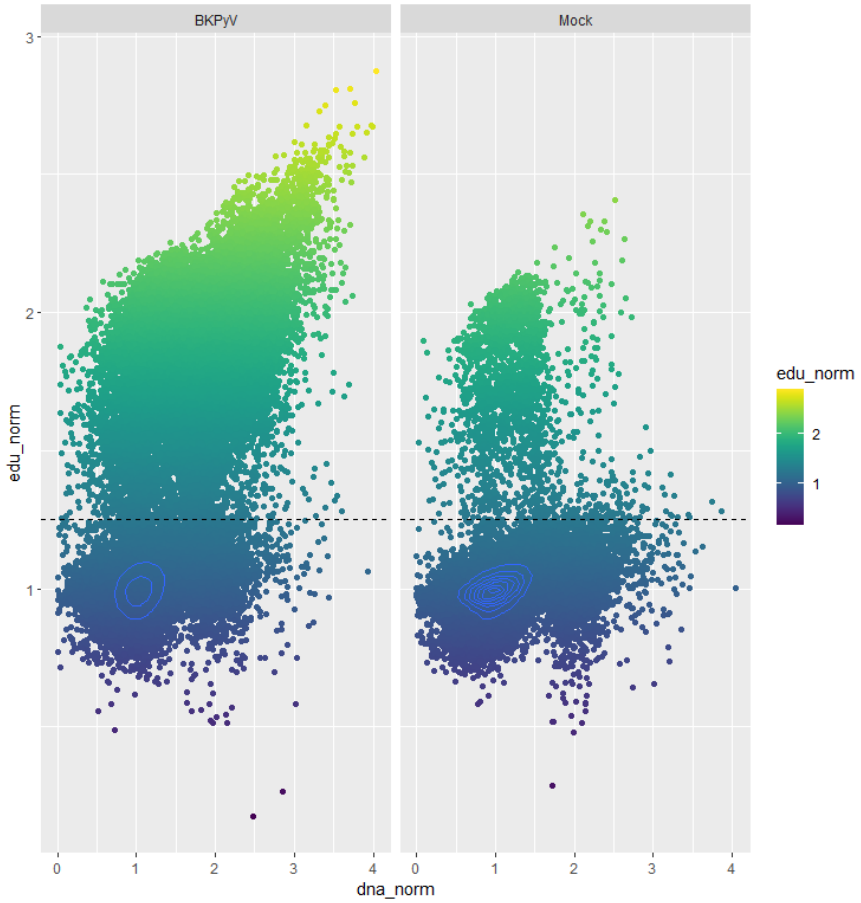


- The script will now generate another line plot, this time with a green vertical line which should intersect the middle of the large peak. You will be asked:

  Is 11.90 representative of the 2N peak? (Y/n/manual)

  If the line is in the middle of the large peak, hit enter. If it isn't, put "n" and input the appropriate value.

- You should now have a new file in your data folder with the suffix "_all_cells.csv". This will be the same as the input file, with a few new columns such as log10-transformed intensity values and an "edu" column.
- The edu column will be called as Positive or Negative for each cell, which has been gated based on your inputs while running SirMixaPlot. You should check this to make sure your gating was accurate. To do so, generate a new dot plot of DNA vs EdU intensity. For example:



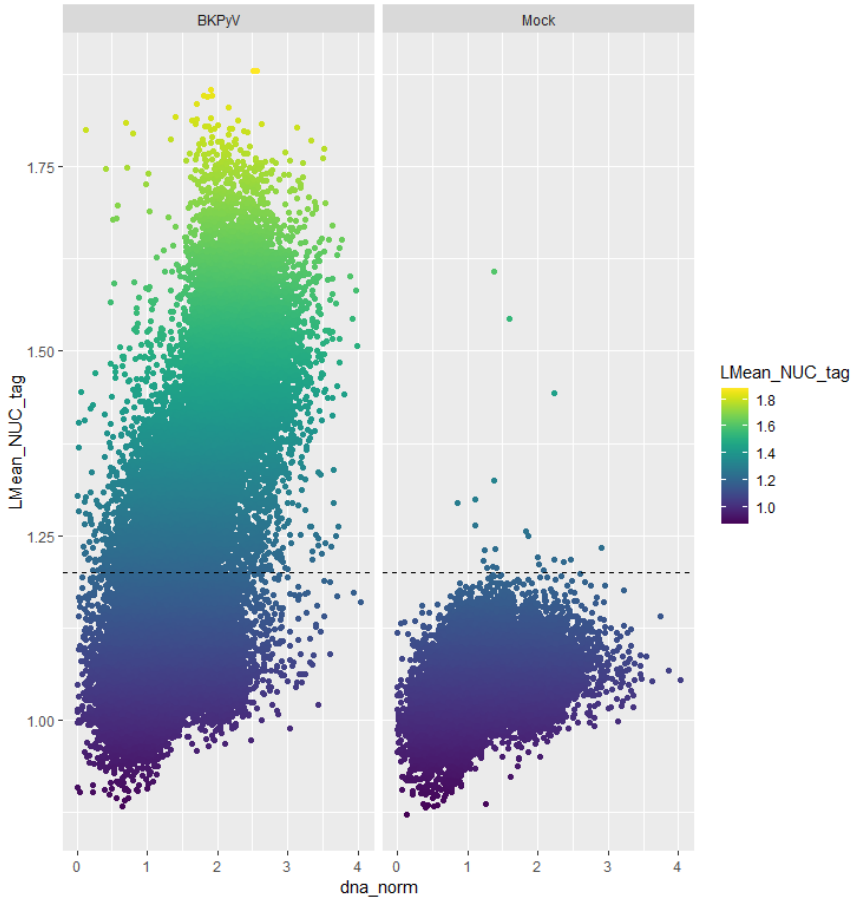
Here I have made a basic dot plot, then faceted by infection and colored by edu_norm value. I have also drawn a dotted line where I gated my EdU positive/negative cells. All of these things can make it easier to visualize the difference between S phase positive and negative cells.

If I wanted to adjust this value, like raise the threshold to 1.3, I could do so by writing something like

datum$edu <- "Negative" (set all rows in the edu column back to negative)

datum[datum$edu_norm > 1.3,]$edu <- "Positive" (set the new threshold for EdU+)

- Setting thresholds for TAg (or anything really) can be done in a similar fashion:



- You nearly always get a few false positives in the Mock samples – this is usually cell debris or other artifacts.
- Once you are done with setting thresholds, save your dataframe as a new .csv file.

Step 10b: Run other analysis scripts to get handy information.

- J. Needham has generated writeups about these scripts which explain them much better than I can. I will just include a few notes on the two analysis scripts that I use the most – explore and icellate.
- The explore script can be used to generate smaller dataframes with info data such as cell counts, % TAg positivity, and cell cycle states.
- To use explore, you must input variables such as
    - FileName: the name of the file to be analyzed
    - Categories (T/F): will you be splitting your data by any variables?
    - cellCycle (T/F): do you want the cell cycle information to be included in your resulting file?
    - saveFile: the name you'd like to give the results file (default is experiment_explored.csv)
- For example, if I'd like to only look at cell cycle states, I could run something like

```
explore(fileName = "./data/experiment_all_cells.csv",

categories = F,
```

```
cellCycle = T,

saveFile = "./data/sphase_explored.csv")
```

This will generate a .csv file with the % of cells in each cell cycle state for each condition in the experiment.

- Or, if I'd like to compare the cell cycle states of TAg positive cells, I could do

```
explore(fileName = "./data/experiment_all_cells.csv",

categories = T,

cellCycle = T,

saveFile = "./data/tag_explored.csv")
```

The script will prompt me to input how many variables ("categories") to split by and name them. Here, I would say 1 category and then input "tag" or whatever I named the column for TAg negative/positive cells. This will generate a .csv file with the cell cycle info, further split up by whether the cells are TAg positive or negative.

- The icellate script can be used to randomly select cells and save those images to a new folder. This is useful for checking your gating (select positive and negative cells) or to get a representative image of a cell in the condition of your choosing.
- I have pasted images of Jason's writeups below, in case more details are needed.

```
explore(
        fileName = cells,                  # Name of the dataset to parse
        sortBy = "name_id",                # Categorical variable by which the dataset gets split and
                                           quantified
        categories = T,                    # Whether (T) or not (F) the data is further split by internal
                                           categorical variables. If True, the user will be asked to
                                           choose any number of categorical variables to further split
                                           the data
        cellCycle = T,                     # Whether (T) or not (F) cell cycle data is quantified. `edu`
                                           and `ploidy` must be present as categorical variables for
                                           this to work
        scheme = "schema.csv",             # Location of the schema file so that metadata can be
                                           pulled
        icellate = F,                      # Whether (integer) or not (F) to run icellate. Based on the
                                           `sortBy` and `categories` variables, this will pull n images,
                                           where n is the integer supplied
        icellate_heatMap = T,              # Whether (T) or not (F) to generate viridis heatmaps of
                                           the icellated images. Default is the "inferno" palette. If
                                           you'd prefer a different palette, either run icellate
                                           separately or change the icellate() default palette.
        vSize = F,                         # Whether (T) or not (F) the icellate function should verify
                                           the default image size using the "overlay.png" image
        random = T,                        # Whether (T) or not (F) the icellate function should pull n
                                           random cells or just the first n cells in the supplied dataset
        extraMetrics = F,                  # Whether (T) or not (F) extra, numerical data should be
                                           quantified. If True, the user will be asked to supply any
                                           number of continuous variables and the mean, median, and
                                           standard deviation of those variables will be added to the
                                           summary data
        saveFile = "data/experiment_explored.csv"){  # The output file name. If this name already
                                           exists, the user is prompted if they would like to
                                           concatenate the datasets. If not, the user can choose a new
                                           name for the data
```

Explore is function to automatically generate quantified data from a supplied dataset. By default, the dataset is split by sample ("name_id"), any metadata supplied in the schema is added, and the total number of ROIs within each sample is quantified. Additionally, the data can be further divided based on any number of categorical variables using the `categories` parameter, which will add the "subsetTotal" and "subset_percent" columns to the final data that gives, as well as columns defining the categorical value of that subpopulation. If the `cellCycle` parameter is True, additional data is added using the `edu` and `ploidy` variables within the dataset. Therefore, it is important that the user has already opened the data in sirmixaplot and gated the cell cycle states using DNA content and EdU to generate the `edu` and `ploidy` variables. Additional metrics can be summarized based on the `extrametrics` parameters, which will generate the mean, median, and standard deviation of the supplied, numerical variable within the subpopulation. The function can also call the icellate() function to generate representative images of the subpopulations. Finally, the data is saved under the `saveFile` parameter. If the given file destination already exists, the user is prompted if they would like to choose a new name to save the data under, or concatenate the data. However, the data must contain the same

---

column names for concatenation to work and, therefore, should have the same `categories`, `cellCycle`, and `extraMetrics` parameters and variables.

```
icellate (
        targetCells,                    # A dataset of microCyte ROIs that will be parsed
        folderName = "singleCells",     # The name of the directory these images will be stored
        dMultiplier = 0.3,              # How much extra space is added to the radius of the
                                        ROI
        verifySize = T,                 # Whether or not the user is prompted to verify the
                                        current dMulitplier is appropriate using the
                                        `verifyImage` ID
        fuse = F,                       # Whether to generate a 'merged' image of the collected
                                        icellates
        marked = F,                     # Whether (T) or not (F) to generate additional icellate
                                        images that add a small '+' to the central ROI
        heatIntensity = F,              # Whether to generate an additional version of each
                                        image using the `heatMap_image` function
        heatColor = "inferno",          # Which viridis palette to use
        verifyImage = "overlay.png",    # Which image is used to verify the correct dMultiplier is
                                        being used to pull icellates
        randomize = T,                  # Whether ROIs are pulled from the target dataset
                                        randomly or in order
        samplingNumber = 5,             # The number of icellates to be pulled from the dataset
        lineAnalyses = T,               # whether to generate line analyses from the pulled
                                        icellates
        numberOfLines = 2,              # The number of line analyses to be run for each icellate.
                                        The angles used for these are determined by dividing 360
                                        by the number of lines desired, and can be offset by the
                                        `initialAngle` variable.
        lineGraphs = T,                 # Whether to make the pixel-by-pixel graph of the line
                                        analyses
        initialAngle = 0,               # The number of degrees to offset the initial angles for
                                        line analyses.
        randomSamplerDenom = 10)        # Determines the number of pixels sampled for each
                                        icellate for generating a sampled set of pixels for non-
                                        line based pixel-by-pixel correlations.
```

icellate is a super helpful ability to recall images of specific ROIs, as well as generate line analyses of the various images within each image folder. Since each ROI has a specific directory, image, and x/y coordinate, any ROI in the dataset can be recalled from its original image automatically. This can be done on any subset of ROIs although it is typically used to randomly sample gated populations to confirm the validity of the gating. Furthermore, images can be 'fused', or averaged, to generate a more representative image. While this is, by design, a low resolution product, it is helpful for showing real, averaged images rather than cherry picked or biased images.

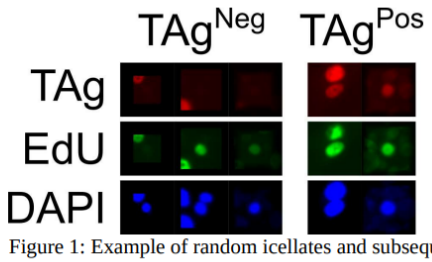


Figure 1: Example of random icellates and subsequent fuse of 10 randomly selected images.

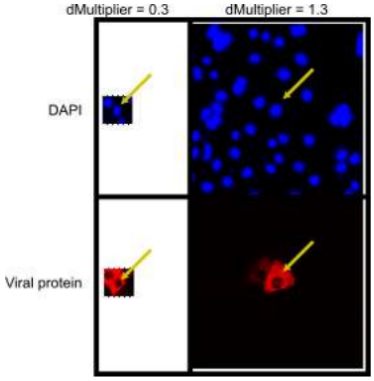


Figure 2: Example of default dMultiplier (0.3) and larger dMultiplier